

InsightFlow - Project Documentation

InsightFlow - Project Documentation

InsightFlow: Autonomous Research Studio - Implementation Guide

Project Overview

InsightFlow is a Streamlit application designed to perform autonomous web research and generate structured reports. It orchestrates a linear pipeline of AI agents (Planner, Researcher, Writer) using Google Gemini and DuckDuckGo.

Project Structure

```
insight_flow/  
  .env                # API Keys (GOOGLE_API_KEY)  
  requirements.txt     # Dependencies  
  app.py              # Main Streamlit UI & Orchestration  
  researcher.py        # Agent 1 & 2: Planning & Research Logic  
  writer.py           # Agent 3: Content Generation Logic  
  utils.py            # Helpers: Gemini Client, Search, Scraping
```

Dependencies (`requirements.txt`)

```
streamlit  
google-generativeai  
duckduckgo-search  
beautifulsoup4  
requests  
python-dotenv
```

Source Code Implementation

1. Utilities (`utils.py`)

Handles LLM interaction (Gemini), Web Search (DuckDuckGo), and Web Scraping.

```
import os  
from duckduckgo_search import DDGS  
import google.generativeai as genai  
import requests  
from bs4 import BeautifulSoup  
from urllib.parse import urlparse  
  
class GeminiProcessor:  
    def __init__(self, api_key):  
        genai.configure(api_key=api_key)  
        self.model = genai.GenerativeModel('gemini-pro')  
  
    def generate_response(self, prompt, temperature=0.7):  
        try:  
            response = self.model.generate_content(  
                prompt,  
                generation_config=genai.types.GenerationConfig(  
                    temperature=temperature
```

InsightFlow - Project Documentation

```
        temperature=temperature,
        max_output_tokens=4096,
    )
)
return response.text
except Exception as e:
    return f"Error: {str(e)}"

def search_web(query, max_results=5):
    try:
        results = []
        with DDGS() as ddgs:
            for r in ddgs.text(query, max_results=max_results):
                results.append(r)
        return results
    except Exception:
        return []

def scrape_website(url):
    try:
        headers = {'User-Agent': 'Mozilla/5.0...'} # Use a real UA string
        response = requests.get(url, headers=headers, timeout=10)
        soup = BeautifulSoup(response.content, 'html.parser')

        # Cleanup
        for tag in soup(["script", "style", "nav", "footer"]):
            tag.decompose()

        return soup.get_text()[:10000] # Truncate for token limits
    except Exception:
        return ""
```

2. Researcher Agent (`researcher.py`)

Responsible for breaking down the topic into queries, searching, and summarizing findings.

```
import json
import streamlit as st
from utils import search_web, scrape_website

class Researcher:
    def __init__(self, gemini_client):
        self.gemini = gemini_client

    def plan_research(self, topic):
        prompt = f"""
        You are a research planner. Generate 3 specific search queries for: '{topic}'.
        Return ONLY a JSON list of strings. Example: ["query1", "query2"]
        """
        response = self.gemini.generate_response(prompt)
```

InsightFlow - Project Documentation

```
try:
    return json.loads(response.replace("```json", "").replace("```", "").strip())
except:
    return [{"topic} analysis"] # Fallback

def conduct_research(self, queries):
    data = []
    for q in queries:
        with st.status(f"Searching: {q}"):
            results = search_web(q, max_results=3)
            for res in results:
                content = scrape_website(res['href'])
                if len(content) > 500:
                    data.append({"title": res['title'], "url": res['href'], "content":
content})
    return data

def summarize_findings(self, topic, research_data):
    context = ""
    for item in research_data:
        context += f"\nSource: {item['title']}\nContent: {item['content'][:2000]}\n"

    prompt = f"""
    Analyze these findings on '{topic}' and write detailed research notes.
    Highlight key facts and contradictions.
    Context: {context[:25000]}
    """
    return self.gemini.generate_response(prompt, temperature=0.4)
```

3. Writer Agent (`writer.py`)

Takes the research notes and formats them into the final deliverable.

```
class Writer:
    def __init__(self, gemini_client):
        self.gemini = gemini_client

    def write_content(self, topic, research_notes, content_type, persona):
        prompt = f"""
        You are a {persona} writer.
        Task: Write a {content_type} on "{topic}".
        Use these notes: {research_notes}

        Structure: Title, Intro, Body, Conclusion.
        Use Markdown.
        """
        return self.gemini.generate_response(prompt, temperature=0.7)
```

4. Main Application (`app.py`)

InsightFlow - Project Documentation

Streamlit interface wiring everything together.

```
import streamlit as st
import os
from dotenv import load_dotenv
from utils import GeminiProcessor
from researcher import Researcher
from writer import Writer

st.set_page_config(page_title="InsightFlow", page_icon="", layout="wide")
load_dotenv()

def main():
    st.title(" InsightFlow: Autonomous Research")

    # Sidebar
    api_key = st.sidebar.text_input("Gemini API Key", type="password")
    content_type = st.sidebar.selectbox("Type", ["Report", "Blog", "Summary"])
    persona = st.sidebar.selectbox("Style", ["Professional", "Academic", "Simple"])

    if query := st.text_input("Research Topic:"):
        if st.button("Start"):
            gemini = GeminiProcessor(api_key)
            researcher = Researcher(gemini)
            writer = Writer(gemini)

            with st.status("Agents working...") as status:
                # 1. Plan
                queries = researcher.plan_research(query)
                st.write(f"Strategy: {queries}")

                # 2. Research
                raw_data = researcher.conduct_research(queries)

                # 3. Summarize
                notes = researcher.summarize_findings(query, raw_data)

                # 4. Write
                report = writer.write_content(query, notes, content_type, persona)
                status.update(label="Done!", state="complete")

            st.markdown(report)
            with st.expander("View Raw Notes"):
                st.write(notes)

if __name__ == "__main__":
    main()
```